Multithreading

Multithreading in Python is a technique that enables a program to execute multiple parts of a task (called threads) *concurrently* within a single process.

Multitasking

Executing Several tasks simultaneously

There are 2 types of Multi-Tasking

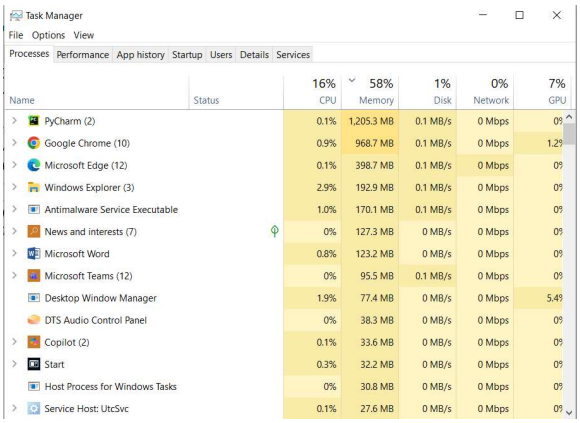
1) Process based Multi-Tasking

2) Thread based Multi-Tasking

1) Process based Multi-Tasking

Executing several tasks simultaneously where each task is a separate independent process is called process based multi-tasking.

Real Time Example:
while typing python program in the editor we can listen mp3 audio songs from the same system.
At the same time, we can download a file from the internet.
All these takes are executing simultaneously and independent of each other.
Hence it is process based multi-tasking.
This type of multi-tasking is best suitable at operating system level.

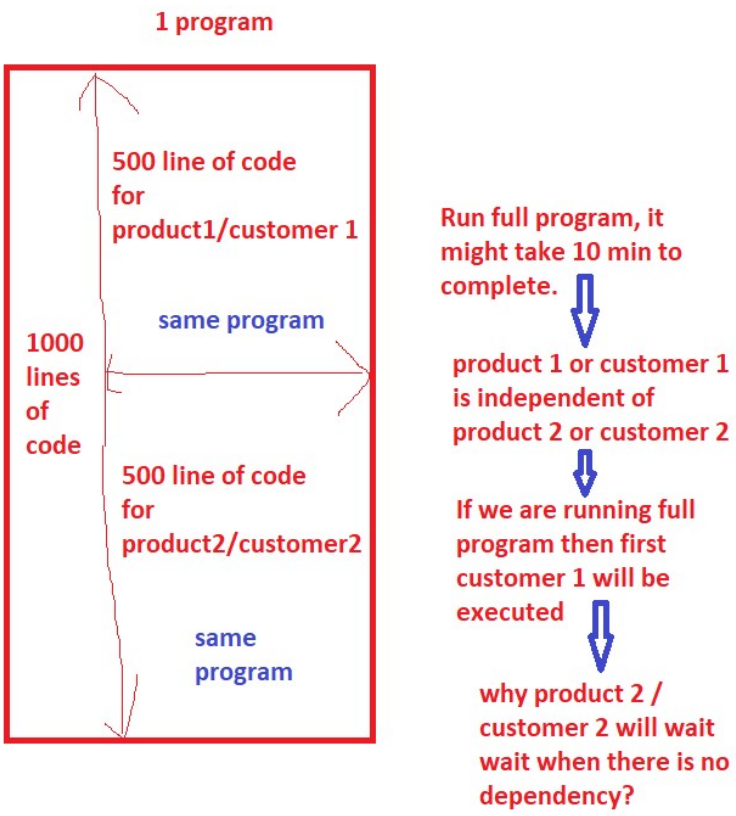


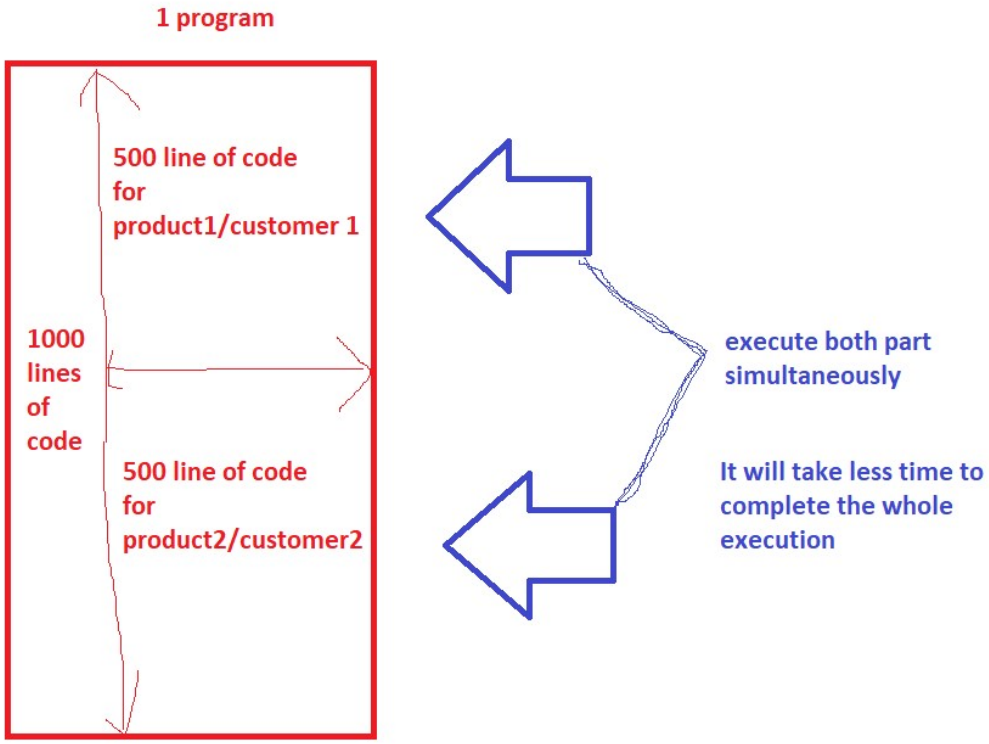
2) Thread based Multi-Tasking

- Executing several tasks simultaneously where each task is a separate independent part of the same program, is called Thread based multi-tasking, and each independent part is called a Thread.

- This type of multi-tasking is best suitable at programmatic level.

Example:





each independent part is called a Thread.

- Flow of execution is considered as Thread.
- Independent part of same program

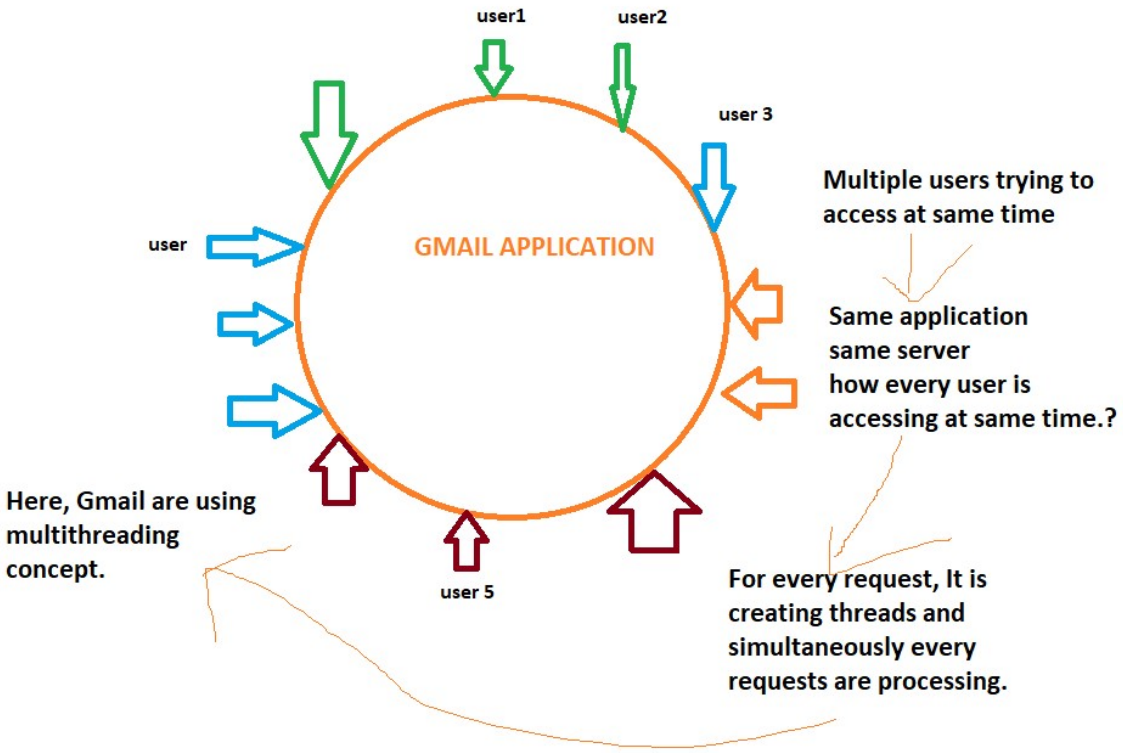
Note:

• Python provides one inbuilt module "threading" to provide support for developing threads. Hence developing multi-threaded Programs is very easy in python.

• Every Python Program by default contains one thread which is nothing but Main Thread.

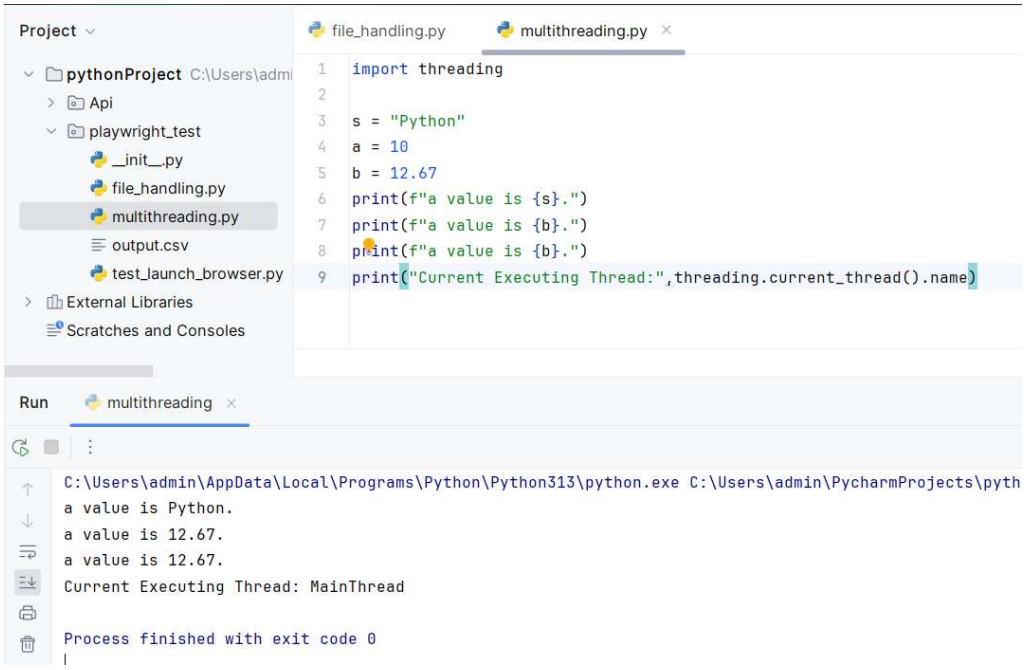
- *Where ever a group of independent jobs are available, then it is highly recommended to execute simultaneously instead of executing one by one.*

Example: -- GMAIL Application



Q) Program to print Name of Current Executing Thread?

```
import threading

s = "Python"
a = 10
b = 12.67
print(f"a value is {s}.")
print(f"a value is {b}.")
print(f"a value is {b}.")
print("Current Executing Thread:",threading.current_thread().name)
```



The ways of Creating Thread in Python:

We can create a thread in Python by using 3 ways

1) Creating a Thread without using any class

2) Creating a Thread by extending Thread class

3) Creating a Thread without extending Thread class

1) Creating a Thread without using any class

```
from threading import *
def display():
    for i in range(1,11):
        print("Child Thread")

t=Thread(target=display) #creating Thread object
t.start() #starting of Thread
for i in range(1,11):
    print("Main Thread")
```

test.py file

```
from threading import *
def display():
    for i in range(1,11):
        print("Child Thread")


t=Thread(target=display) #creating Thread object
t.start() #starting of Thread


for i in range(1,11):
    print("Main Thread")
```

using threading module

We don't know which thread is running when, So we will get random output.

test.py file execution responsiblity with Main thread which is default thread

we have two for loops, we want to execute one with child thread and another with main thread

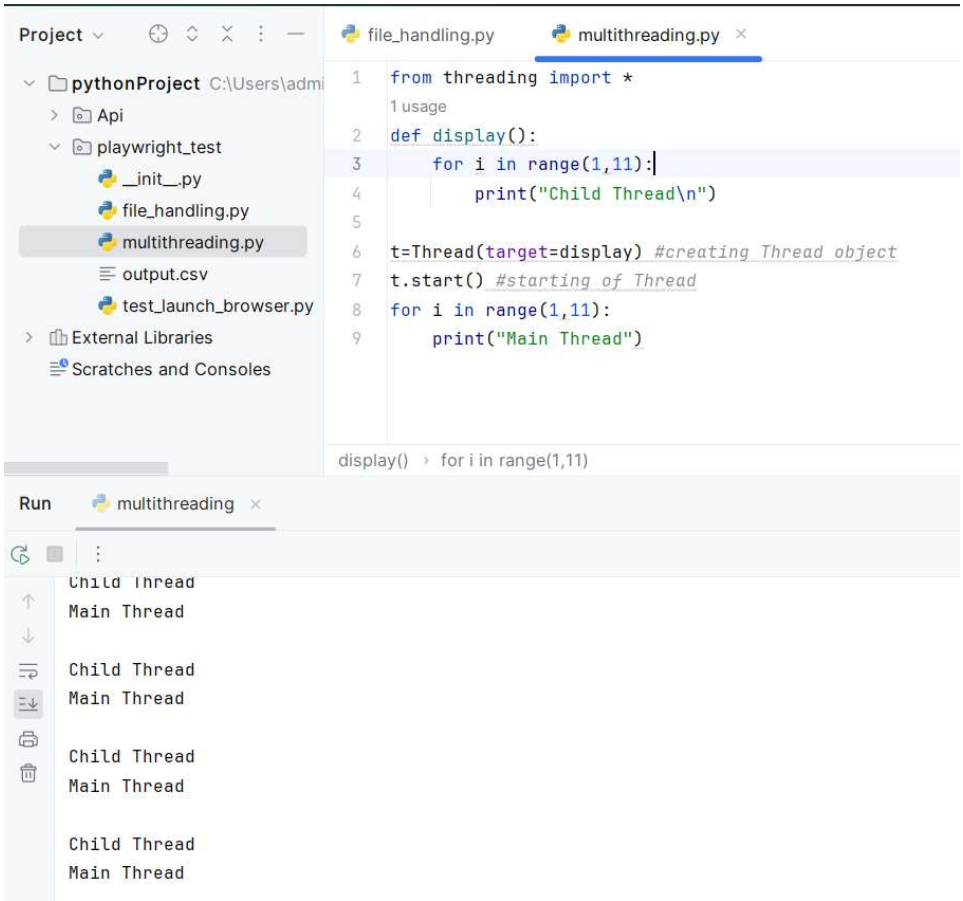
We are creating child thread, Main thread responsiblity is to create child thread.

t=Thread(target=display) *#creating Thread object , here display is the function written inside program.*
t.start() *#starting of Thread*

** From 1st line of code till {t.start()}.....when t.start code executed, above t.start code, child thread responsiblity to execute code*

** After t.start() code, main thread responsiblity to execute code*

Note: Thread is a pre-defined class present in threading module which can be used to create our own Threads.



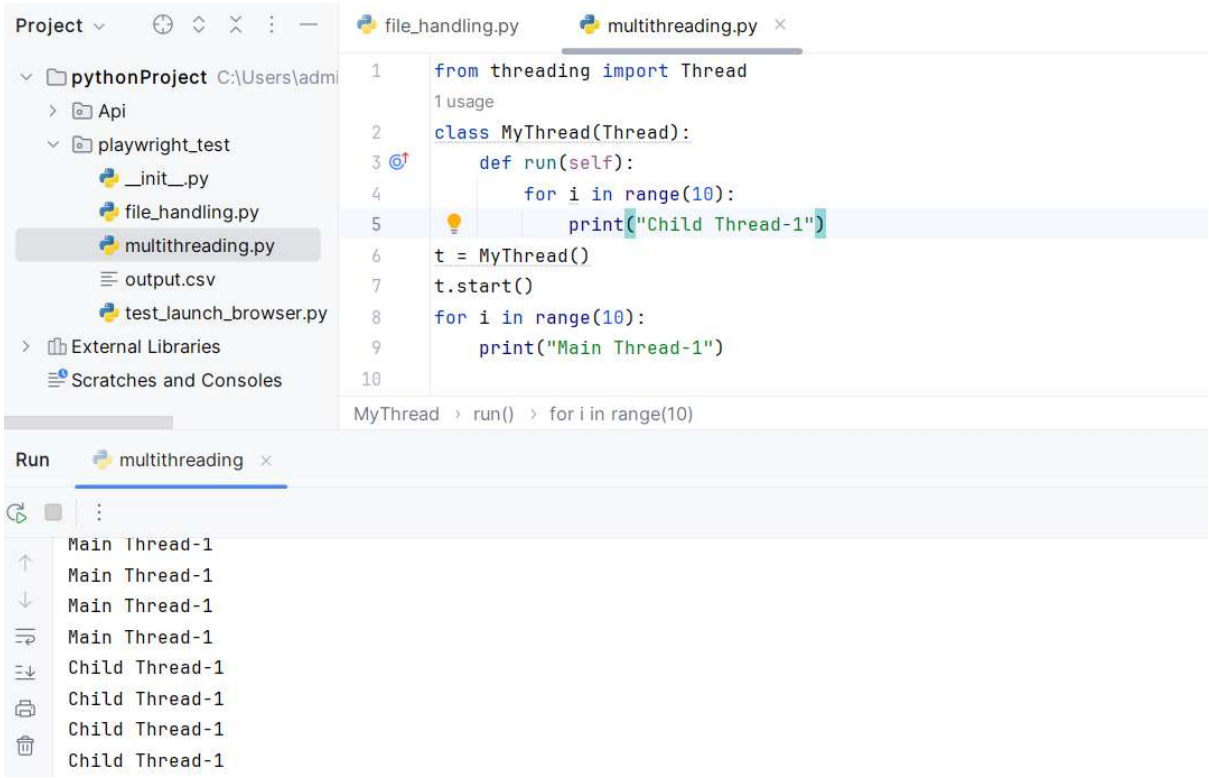
2) Creating a Thread by extending Thread Class

We have to create child class for Thread class.

In that child class we have to override run() method with our required job.

Whenever we call start() method then automatically run() method will be executed and performs our job.

```python
from threading import Thread

class MyThread(Thread):
    def run(self):
        for i in range(10):
            print("Child Thread-1")

t = MyThread()
t.start()

for i in range(10):
    print("Main Thread-1")
```



3) Creating a Thread without extending Thread Class

```
from threading import Thread

class Test:
    def display(self):
        for i in range(10):
            print("Child Thread-2")

obj = Test()
# Passing the instance method as the target
t = Thread(target=obj.display)
t.start()

for i in range(10):
    print("Main Thread-2")
```

Project

- pythonProject C:\Users\admi
  - Api
  - playwright_test
    - __init__.py
    - file_handling.py
    - multithreading.py
    - output.csv
    - test_launch_browser.py
- External Libraries
- Scratches and Consoles

file_handling.py | multithreading.py

```python
from threading import Thread

class Test:
    def display(self):
        for i in range(10):
            print("Child Thread-2")

obj = Test()
# Passing the instance method as the target
t = Thread(target=obj.display)
t.start()

for i in range(10):
    print("Main Thread-2")
```

Test › display()

Run multithreading

```
Main Thread-2Child Thread-2

Main Thread-2Child Thread-2

Main Thread-2Child Thread-2

Main Thread-2Child Thread-2


Process finished with exit code 0
```

Example with & without multithreading

Without multithreading

```
import time

def doubles(numbers):
    for n in numbers:
        time.sleep(1)
        print(f"Double of {n} is {2 * n}.\n")

def squares(numbers):
    for n in numbers:
        time.sleep(1)
        print(f"Square of {n} is {n * n}.\n")

numbers = [1, 2, 3, 4, 5, 6]
begin_time = time.time()
doubles(numbers)
squares(numbers)
print("The total time taken:", time.time() - begin_time)
```

```
C:\Users\admin\AppData\Local\Programs\Python\Python313\python.exe
Double of 1 is 2.

Double of 2 is 4.

Double of 3 is 6.

Double of 4 is 8.

Double of 5 is 10.

Double of 6 is 12.

Square of 1 is 1.

Square of 2 is 4.

Square of 3 is 9.

Square of 4 is 16.

Square of 5 is 25.

Square of 6 is 36.

The total time taken: 12.007773637771606

Process finished with exit code 0
```

With Multithreading

```python
import threading
import time

def doubles(numbers):
    for n in numbers:
        time.sleep(1)
        print(f"Double of {n} is {2 * n}.\n")

def squares(numbers):
    for n in numbers:
        time.sleep(1)
        print(f"Square of {n} is {n * n}.\n")

numbers = [1, 2, 3, 4, 5, 6]
begin_time = time.time()

# Create the threads
t1 = threading.Thread(target=doubles, args=(numbers,))
t2 = threading.Thread(target=squares, args=(numbers,))

# Start the threads
t1.start()
t2.start()

# Wait for both to finish before moving to the final print
t1.join()
t2.join()

print("The total time taken:", time.time() - begin_time)
```

```
C:\Users\admin\AppData\Local\Programs\Python\Python313\python.exe
Double of 1 is 2.

Square of 1 is 1.

Double of 2 is 4.

Square of 2 is 4.

Double of 3 is 6.

Square of 3 is 9.

Double of 4 is 8.

Square of 4 is 16.

Double of 5 is 10.

Square of 5 is 25.

Double of 6 is 12.

Square of 6 is 36.

The total time taken: 6.006454944610596

Process finished with exit code 0
```

Thread Identification Number (ident):

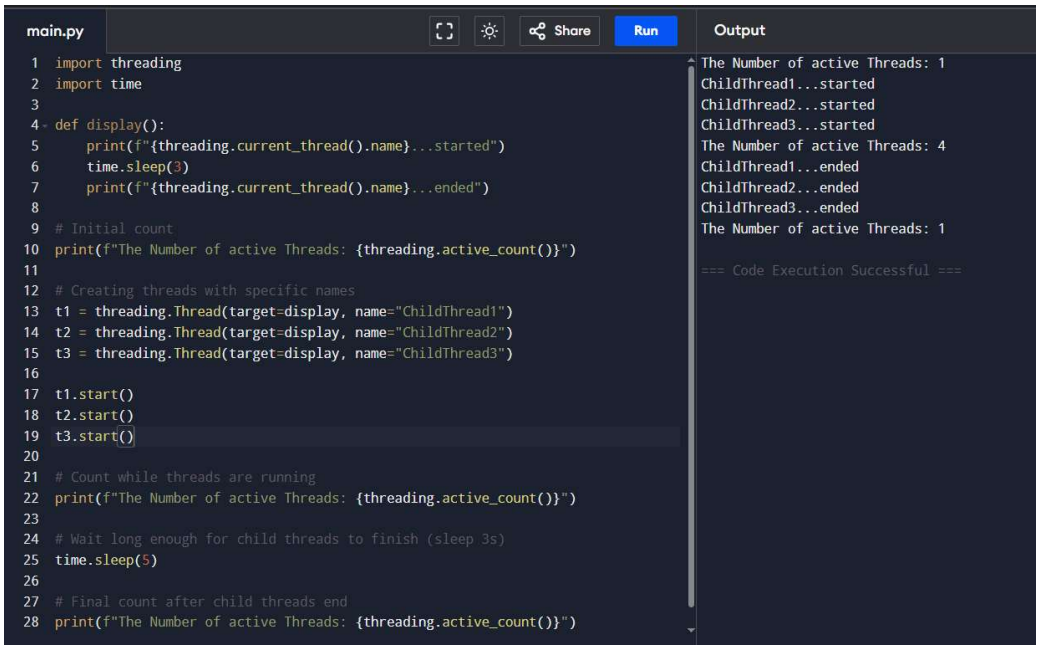
For every thread internally a unique identification number is available. We can access this id by using implicit variable "ident".

```python
from threading import Thread, current_thread
import time
def test():
    print("Child Thread running...")

t = Thread(target=test)

# This will return None because the thread hasn't started yet
print("Child ID before start:", t.ident)

t.start()


# Now it will show the ID
print("Main Thread ID:", current_thread().ident)
print("Child Thread ID:", t.ident)
```

active_count():

This function returns the number of active threads currently running.

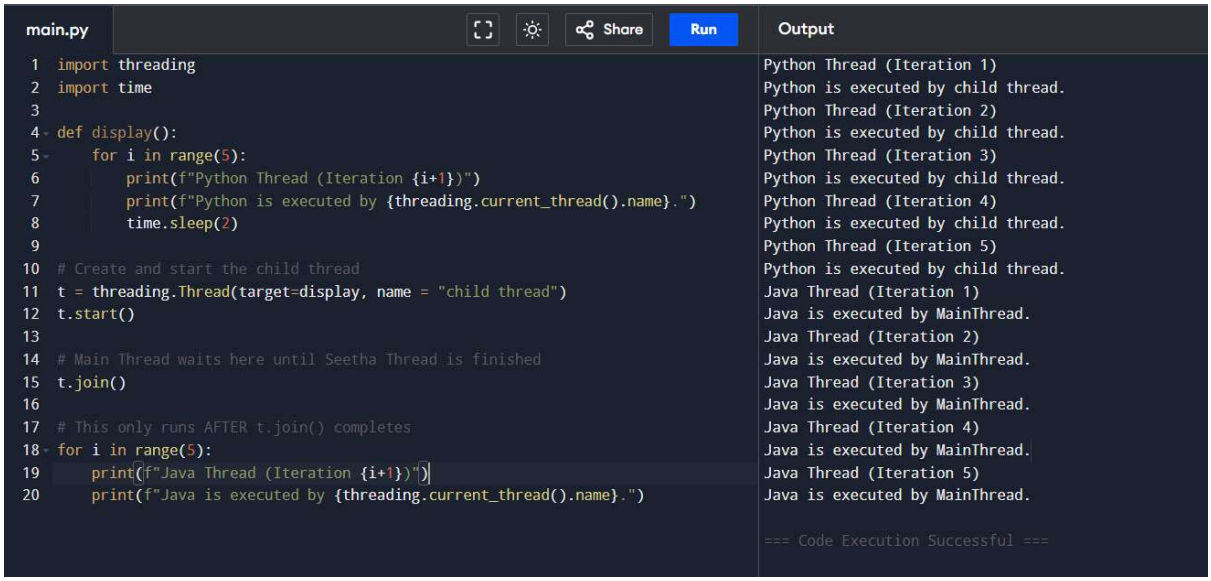
```python
import threading
import time

def display():
    print(f"{threading.current_thread().name}...started")
    time.sleep(3)
    print(f"{threading.current_thread().name}...ended")

# Initial count
print(f"The Number of active Threads: {threading.active_count()}")

# Creating threads with specific names
t1 = threading.Thread(target=display, name="ChildThread1")
t2 = threading.Thread(target=display, name="ChildThread2")
t3 = threading.Thread(target=display, name="ChildThread3")

t1.start()
t2.start()
t3.start()

# Count while threads are running
print(f"The Number of active Threads: {threading.active_count()}")

# Wait long enough for child threads to finish (sleep 3s)
time.sleep(5)

# Final count after child threads end
print(f"The Number of active Threads: {threading.active_count()}")
```



join() Method:

If a thread wants to wait until completing some other thread then we should go for join() method.

```python
import threading
import time

def display():
    for i in range(5):
        print(f"Python Thread (Iteration {i+1})")
        print(f"Python is executed by {threading.current_thread().name}.")
        time.sleep(2)

# Create and start the child thread
t = threading.Thread(target=display, name = "child thread")
t.start()

# Main Thread waits here until Seetha Thread is finished
t.join()

# This only runs AFTER t.join() completes
for i in range(5):
    print(f"Java Thread (Iteration {i+1})")
    print(f"Java is executed by {threading.current_thread().name}.")
```



IF we will not use join() method then

Both thread will run simultaneously and random output will appear.

main.py

```python
import threading
import time

def display():
    for i in range(5):
        print(f"Python Thread (Iteration {i+1})")
        print(f"Python is executed by {threading.current_thread().name}.")
        time.sleep(2)

# Create and start the child thread
t = threading.Thread(target=display, name = "child thread")
t.start()

# Main Thread waits here until Seetha Thread is finished
#t.join()

# This only runs AFTER t.join() completes
for i in range(5):
    print(f"Java Thread (Iteration {i+1})")
    print(f"Java is executed by {threading.current_thread().name}.")
```

Output

```
Python Thread (Iteration 1)
Python is executed by child thread.
Java Thread (Iteration 1)
Java is executed by MainThread.
Java Thread (Iteration 2)
Java is executed by MainThread.
Java Thread (Iteration 3)
Java is executed by MainThread.
Java Thread (Iteration 4)
Java is executed by MainThread.
Java Thread (Iteration 5)
Java is executed by MainThread.
Python Thread (Iteration 2)
Python is executed by child thread.
Python Thread (Iteration 3)
Python is executed by child thread.
Python Thread (Iteration 4)
Python is executed by child thread.
Python Thread (Iteration 5)
Python is executed by child thread.

=== Code Execution Successful ===
```

isAlive() Method:

isAlive() method checks whether a thread is still executing or not.

```python
import threading
import time

def display():
    # Use .name instead of .getName()
    print(f"{threading.current_thread().name}...started")
    time.sleep(3)
    print(f"{threading.current_thread().name}...ended")

t1 = threading.Thread(target=display, name="ChildThread1")
t2 = threading.Thread(target=display, name="ChildThread2")

t1.start()
t2.start()

# Use .is_alive() instead of .isAlive()
print(f"{t1.name} is Alive: {t1.is_alive()}")
print(f"{t2.name} is Alive: {t2.is_alive()}")

# Wait for threads to finish their 3-second sleep
time.sleep(5)

print(f"{t1.name} is Alive: {t1.is_alive()}")
print(f"{t2.name} is Alive: {t2.is_alive()}")
```

main.py

```python
import threading
import time

def display():
    # Use .name instead of .getName()
    print(f"{threading.current_thread().name}...started")
    time.sleep(3)
    print(f"{threading.current_thread().name}...ended")

t1 = threading.Thread(target=display, name="ChildThread1")
t2 = threading.Thread(target=display, name="ChildThread2")

t1.start()
t2.start()

# Use .is_alive() instead of .isAlive()
print(f"{t1.name} is Alive: {t1.is_alive()}")
print(f"{t2.name} is Alive: {t2.is_alive()}")

# Wait for threads to finish their 3-second sleep
time.sleep(5)

print(f"{t1.name} is Alive: {t1.is_alive()}")
print(f"{t2.name} is Alive: {t2.is_alive()}")
```

Output

```
ChildThread1...started
ChildThread2...started
ChildThread1 is Alive: True
ChildThread2 is Alive: True
ChildThread1...ended
ChildThread2...ended
ChildThread1 is Alive: False
ChildThread2 is Alive: False

=== Code Execution Successful ===
```

Daemon Threads:

- The threads which are running in the background are called Daemon Threads.
- The main objective of Daemon Threads is to provide support for Non Daemon Threads (like main thread)

  Eg: Garbage Collector

- Whenever Main Thread runs with low memory, immediately PVM runs Garbage Collector to destroy useless objects and to provide free memory, so that Main Thread can continue its execution without having any memory problems.
- We can check whether thread is Daemon or not by using

  t.isDaemon() method of Thread class or by using daemon property.

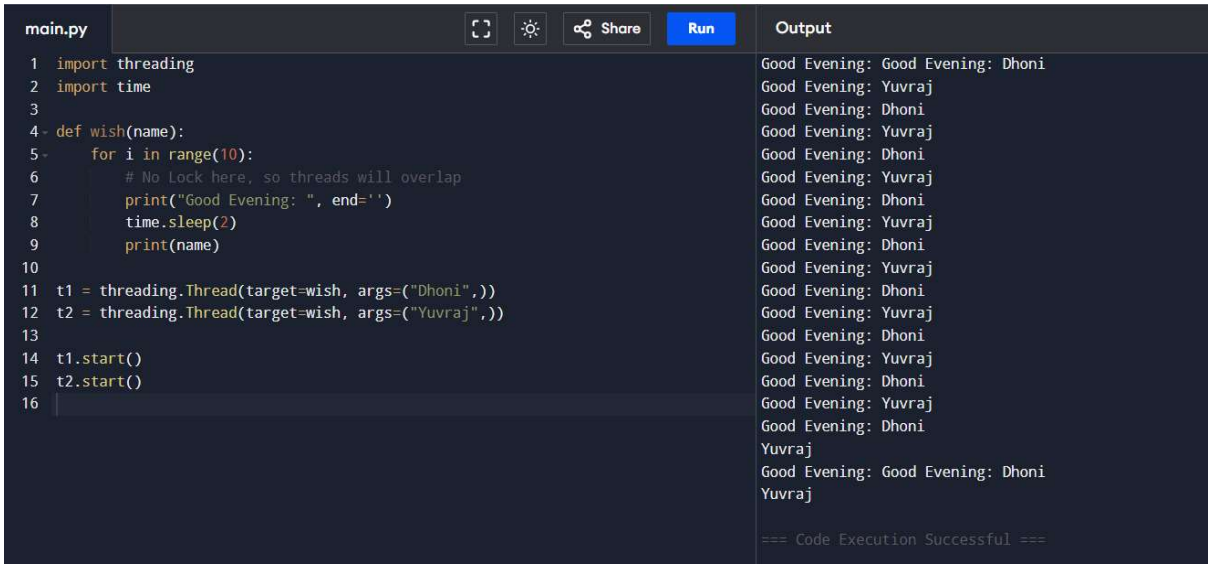
```python
import threading

# Modern way to check if the Main Thread is a daemon
print(f"Is Main Thread daemon? {threading.current_thread().daemon}")

# Output will be False because the Main Thread is never a daemon by default
```

- By default, Main Thread is always non-daemon. But for the remaining threads Daemon nature will be inherited from parent to child. i.e if the Parent Thread is Daemon then child thread is also Daemon and if the Parent Thread is Non Daemon then Child Thread is also Non Daemon.

Synchronization:

If multiple threads are executing simultaneously then there may be a chance of data inconsistency problems.

```
import threading
import time

def wish(name):
    for i in range(10):
        # No Lock here, so threads will overlap
        print("Good Evening: ", end='')
        time.sleep(2)
        print(name)

t1 = threading.Thread(target=wish, args=("Dhoni",))
t2 = threading.Thread(target=wish, args=("Yuvraj",))

t1.start()
t2.start()
```



Why the output looks weird:

1. **Thread 1** prints `"Good Evening: "` and then goes to sleep for 2 seconds.
2. **Thread 2** starts immediately, prints `"Good Evening: "` on the **same line**, and also goes to sleep.
3. After 2 seconds, both wake up and try to print their names at once.
4. You will likely see output like: `Good Evening: Good Evening: Dhoni Yuvraj` instead of two separate lines.

In Python, we can implement synchronization by using the following

1) Lock

2) RLock

3) Semaphore

Synchronization By using Lock Concept:

• Locks are the most fundamental synchronization mechanism provided by threading module.

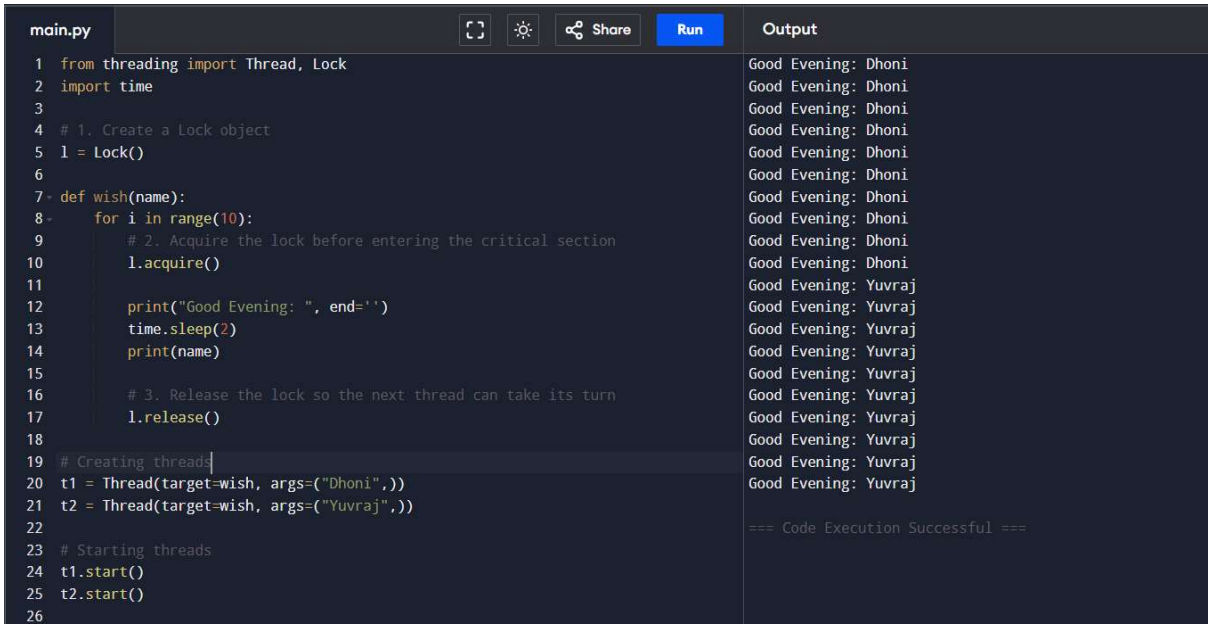
• We can create Lock object as follows l = Lock ()

• The Lock object can be hold by only one thread at a time. If any other thread required, the same lock then it will wait until thread releases lock. (Similar to common wash rooms, public telephone booth etc.)

• A Thread can acquire the lock by using acquire () Method acquire ()

• A Thread can release the lock by using release () Method release ()

```python
from threading import Thread, Lock
import time

# 1. Create a Lock object
l = Lock()


def wish(name):
    for i in range(10):
        # 2. Acquire the lock before entering the critical section
        l.acquire()

        print("Good Evening: ", end='')
        time.sleep(2)
        print(name)

        # 3. Release the lock so the next thread can take its turn
        l.release()


# Creating threads
t1 = Thread(target=wish, args=("Dhoni",))
t2 = Thread(target=wish, args=("Yuvraj",))

# Starting threads
t1.start()
t2.start()
```



Note:

- **above program at a time only one thread is allowed to execute wish() method and hence we will get regular output.**

- **To call release() method compulsory thread should be owner of that lock. i.e thread should has the lock already, otherwise we will get Runtime Exception saying RuntimeError: release unlocked lock**

Synchronization By using RLock Concept:

An **RLock** (Re-entrant Lock) is a special type of lock that allows the **same thread** to acquire it multiple times without getting stuck (deadlocking itself)

In a standard `Lock`, if a thread calls `.acquire()` a second time before calling `.release()`, it will wait forever for itself to release the lock. An `RLock` solves this by keeping track of the "recursion level."

```
from threading import Thread, RLock
import time

# 1. Create an RLock (Re-entrant Lock) object
# It acts like a "Smart Lock" that recognizes the current thread
l = RLock()


def wish(name):
    for i in range(3):
        # 2. Acquire the lock
        l.acquire()

        print("Good Evening: ", end='')
        time.sleep(2)
        print(name)

        # 3. Release the lock
        l.release()


# Creating threads
t1 = Thread(target=wish, args=("Dhoni",))
t2 = Thread(target=wish, args=("Yuvraj",))

# Starting threads
t1.start()
t2.start()
```

Why use `RLock` in this specific code?

In this simple script, `Lock` and `RLock` will produce the **exact same result**. However, `RLock` is safer for developers because:

1. **Avoids Self-Deadlock:** If you accidentally call `l.acquire()` twice in the same thread, the program won't freeze.
2. **Recursive Safety:** If the `wish` function called itself (recursion), a standard `Lock` would hang forever, but `RLock` would keep working.

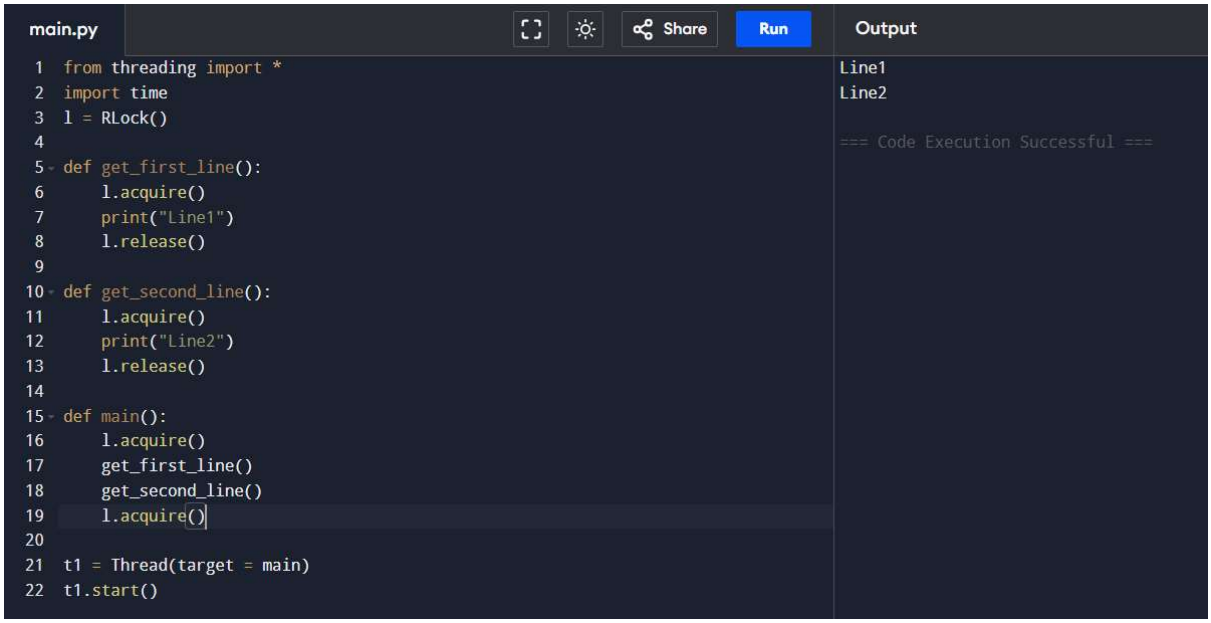
Difference between Lock and Rlock

```
from threading import *
import time

l = Lock()


def get_first_line():
    l.acquire()
    print("Line1")
    l.release()


def get_second_line():
    l.acquire()
    print("Line2")
    l.release()


def main():
    l.acquire()
    get_first_line()
    get_second_line()
    l.acquire()


t1 = Thread(target=main)
t1.start()
```

output → Program crashed.

Why -> two times acquired called…?

```python
from threading import *
import time

l = RLock()


def get_first_line():
    l.acquire()
    print("Line1")
    l.release()


def get_second_line():
    l.acquire()
    print("Line2")
    l.release()


def main():
    l.acquire()
    get_first_line()
    get_second_line()
    l.acquire()


t1 = Thread(target=main)
t1.start()
```



Lock	RLock
1) Lock object can be acquired by only one thread at a time.Even owner thread also cannot acquire multiple times.	1) RLock object can be acquired by only one thread at a time, but owner thread can acquire same lock object multiple times.
2) Not suitable to execute recursive functions and nested access calls.	2) Best suitable to execute recursive functions and nested access calls.
3) In this case Lock object will takes care only Locked or unlocked and it never takes care about owner thread and recursion level.	3) In this case RLock object will takes care whether Locked or unlocked and owner thread information, recursiion level.

Synchronization by using Semaphore:

- For Lock and RLock, at a time only one thread is allowed to execute.
- Sometimes our requirement is at a time a particular number of threads are allowed to access (like at a time 10 members are allowed to access database server,4 members are allowed to access Network connection etc.).
- To handle this requirement, we cannot use Lock and RLock concepts and we should go for Semaphore concept.
- Semaphore can be used to limit the access to the shared resources with limited capacity.
- Semaphore is advanced Synchronization Mechanism.
- We can create Semaphore object as follows s = Semaphore(counter)

- Here counter represents the maximum number of threads are allowed to access simultaneously. The default value of counter is 1.
- Whenever thread executes acquire () method, then the counter value will be decremented by 1 and if thread executes release () method then the counter value will be incremented by 1
- for every acquire () call counter value will be decremented and for every release () call counter value will be incremented.

Case-1:

s = Semaphore ()

In this case counter value is 1 and at a time only one thread is allowed to access. It is exactly same as Lock concept

Case-2:

s = Semaphore (3) In this case Semaphore object can be accessed by 3 threads at a time. The remaining threads have to wait until releasing the semaphore.

```python
import threading
import time

# Semaphore(2) allows 2 threads to execute the critical section at the same time
s = threading.Semaphore(2)

def wish(name):
    # Acquire a permit from the semaphore
    s.acquire()
    for i in range(10):
        # .name replaces the deprecated getName()
        print(f"Good Evening {threading.current_thread().name}: ", end='')
        time.sleep(2)
        print(name)
    # Release the permit for the next waiting thread
    s.release()

# Creating 5 threads
names = ["Dhoni", "Yuvraj", "Kohli", "Rohit", "Pandya"]
threads = []

for name in names:
    t = threading.Thread(target=wish, args=(name,), name=f"Thread-{name}")
    threads.append(t)
    t.start()

# Optional: Join threads to wait for all to finish
for t in threads:
    t.join()
```

main.py

Share Run

```python
import threading
import time

# Semaphore(2) allows 2 threads to execute the critical section at the same time
s = threading.Semaphore(2)

def wish(name):
    # Acquire a permit from the semaphore
    s.acquire()
    for i in range(4):
        # .name replaces the deprecated getName()
        print(f"Good Evening {threading.current_thread().name}: ", end='')
        time.sleep(2)
        print(name)
    # Release the permit for the next waiting thread
    s.release()

# Creating 5 threads
names = ["Dhoni", "Yuvraj", "Kohli", "Rohit", "Pandya"]
threads = []

for name in names:
    t = threading.Thread(target=wish, args=(name,))
    threads.append(t)
    t.start()

# Optional: Join threads to wait for all to finish
```

Output

```
Good Evening Thread-1 (wish): Good Evening Thread-2 (wish): Dhoni
Good Evening Thread-1 (wish): Yuvraj
Good Evening Thread-2 (wish): Dhoni
Good Evening Thread-1 (wish): Yuvraj
Good Evening Thread-2 (wish): Dhoni
Good Evening Thread-1 (wish): Yuvraj
Good Evening Thread-2 (wish): Dhoni
Yuvraj
Good Evening Thread-3 (wish): Good Evening Thread-4 (wish): Kohli
Rohit
Good Evening Thread-3 (wish): Good Evening Thread-4 (wish): Kohli
Good Evening Thread-3 (wish): Rohit
Good Evening Thread-4 (wish): Kohli
Good Evening Thread-3 (wish): Rohit
Good Evening Thread-4 (wish): Kohli
Rohit
Good Evening Thread-5 (wish): Pandya
Good Evening Thread-5 (wish): Pandya
Good Evening Thread-5 (wish): Pandya
Good Evening Thread-5 (wish): Pandya

=== Code Execution Successful ===
```